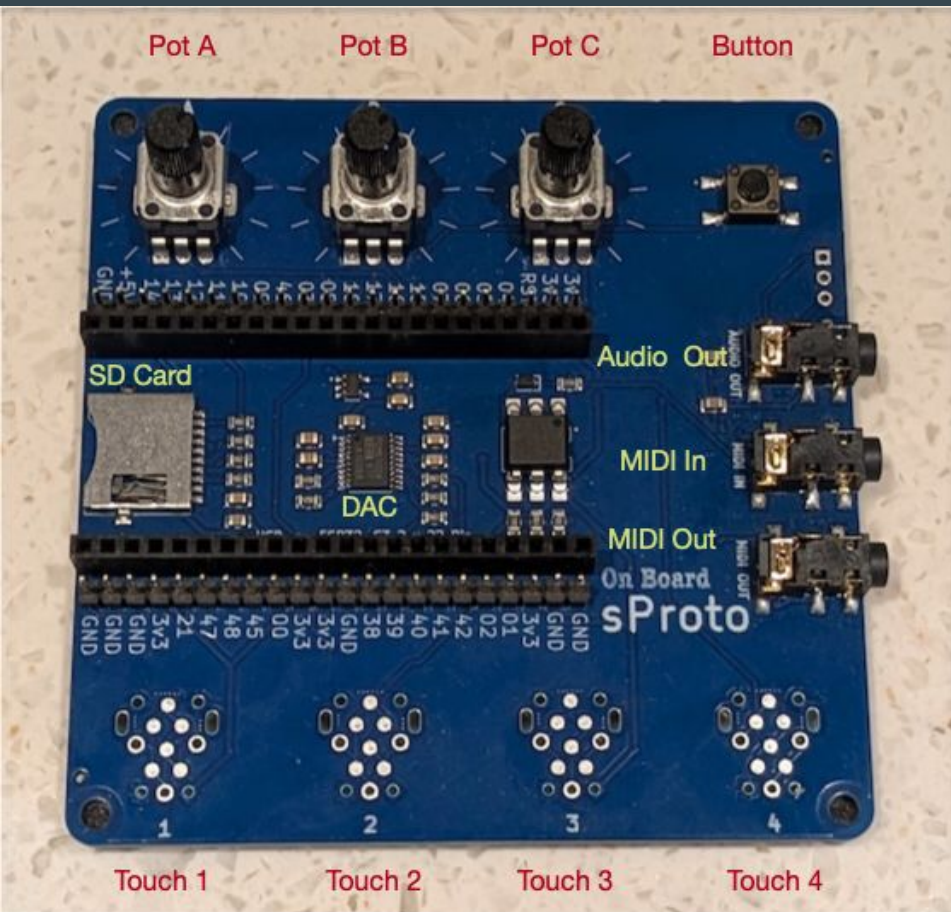


Coding a Digital Sound Box

...

On the ESP32 with the sProto PCB & Arduino IDE



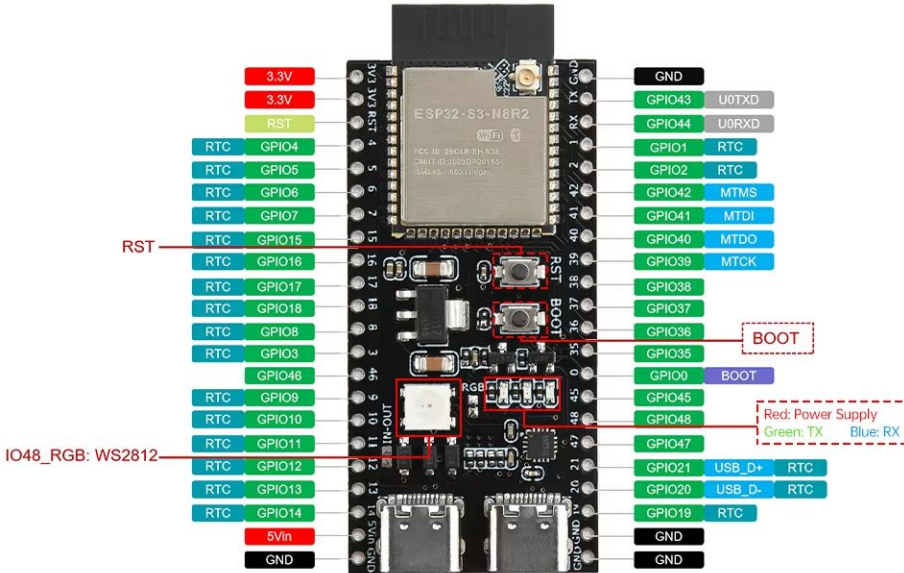
The sProto PCB

PCB for ESP32-S3 Development Board

Features:

- Pin Headers & Breakout Pins
 - UI Controls (Pots, Button, Pads)
 - I2S stereo audio output
 - TRS A MIDI in and out
 - Mini SD Card reader
-

ESP32-S3 Dev Board



- A dual-core Xtensa LX7 MCU, running at 240 MHz.
- Integrated 2.4 GHz, 802.11 b/g/n Wi-Fi and Bluetooth 5 (LE)
- 512 KB of SRAM, 8 MB Flash, 16 MB PSRAM
- 2 x 22 pin headers
- “ESP32-S3-DevKitC-1” on Aliexpress or Ebay or Amazon

Playing with the sProto Sound Box



sProto Sound Box

Four independent instruments on one sProto board:

- Pad 1 / Dial 1: Tin Whistle, Trigger / Pitch
- Pad 2 / Dial 2: Kalimba, Trigger / Pitch
- Pad 3 / Dial 3: Spring, Trigger / Multiple Triggers
- Pad 4: Glockenspiel, Trigger

Hold the sProto button to adjust effects:

- Dial 1: Delay Time
- Dial 2: Delay Level
- Dial 3: Reverb Level

Introducing Coding Tools

- Arduino IDE

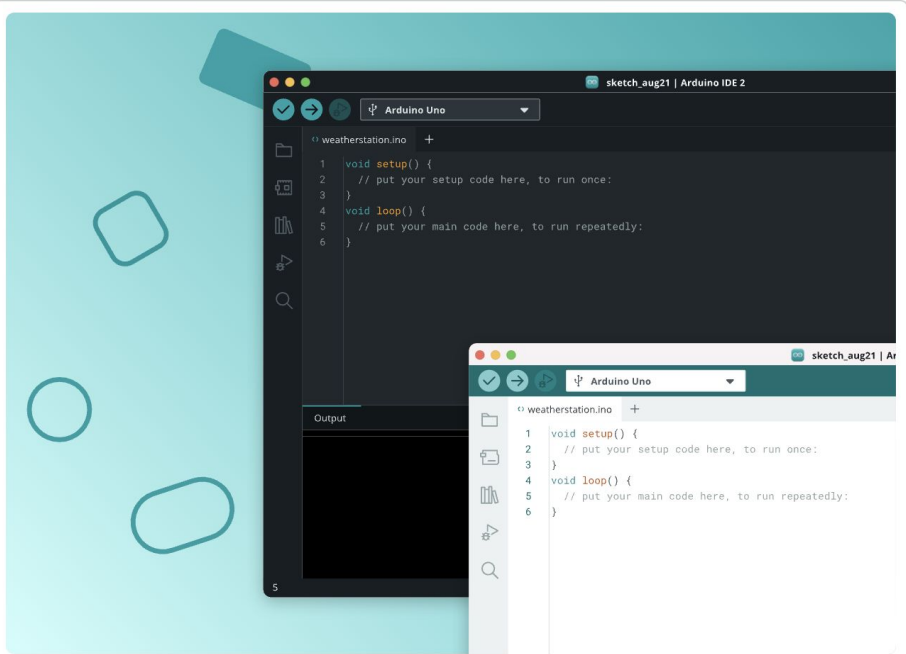
- M16 Synthesis Library

- MultiControl Library

```
16 #include "M16.h"
17 #include "Osc.h"
18 #include "Env.h"
19 #include "SVF2.h"
20 #include "SVF.h"
21 #include "Phys.h"
22 #include "Rel.h"
23 #include "E.h"
24 #include "FX.h"
25 #if IS_ESP32()
26     #include <esp32/Arduino.h>
27     #include <esp_system.h>
28 #endif
29
30 // -----
31 // Shared effects and timing
32
33 BBD bbdDelay;
34 volatile bool bbdDelayComplete = false;
35 const float minBbdDelayMs = 4096.0f / 44100.0f * 1000.0f / 3.0f;
36 const float maxBbdDelayMs = 2000.0f;
37 const float defaultBbdDelayMs = 200.0f;
38
39 unsigned long controlTime = 0;
40 unsigned long midiTime = 0;
```



Bring Your Projects to Life with Arduino Software

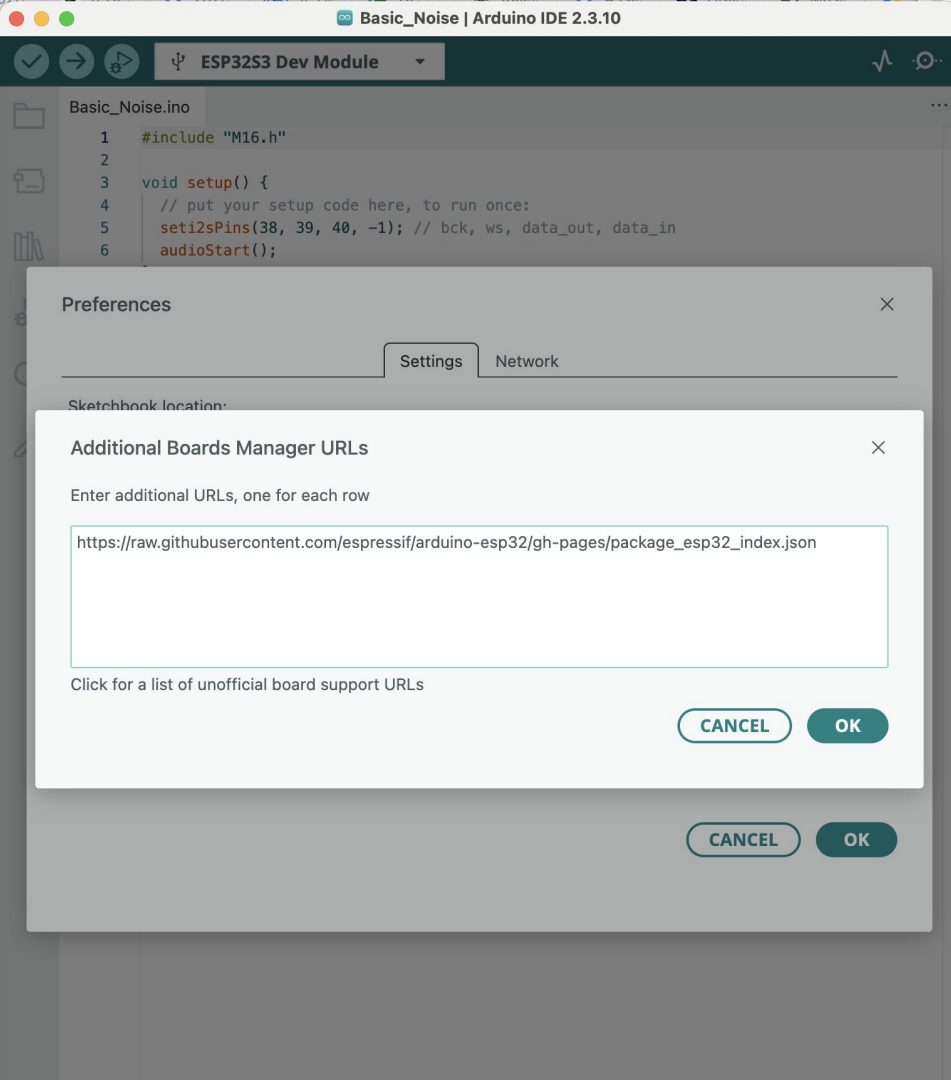


Arduino IDE 2.3.10
Release notes

Install the IDE

Integrated Development Environment

<https://www.arduino.cc/en/software/>



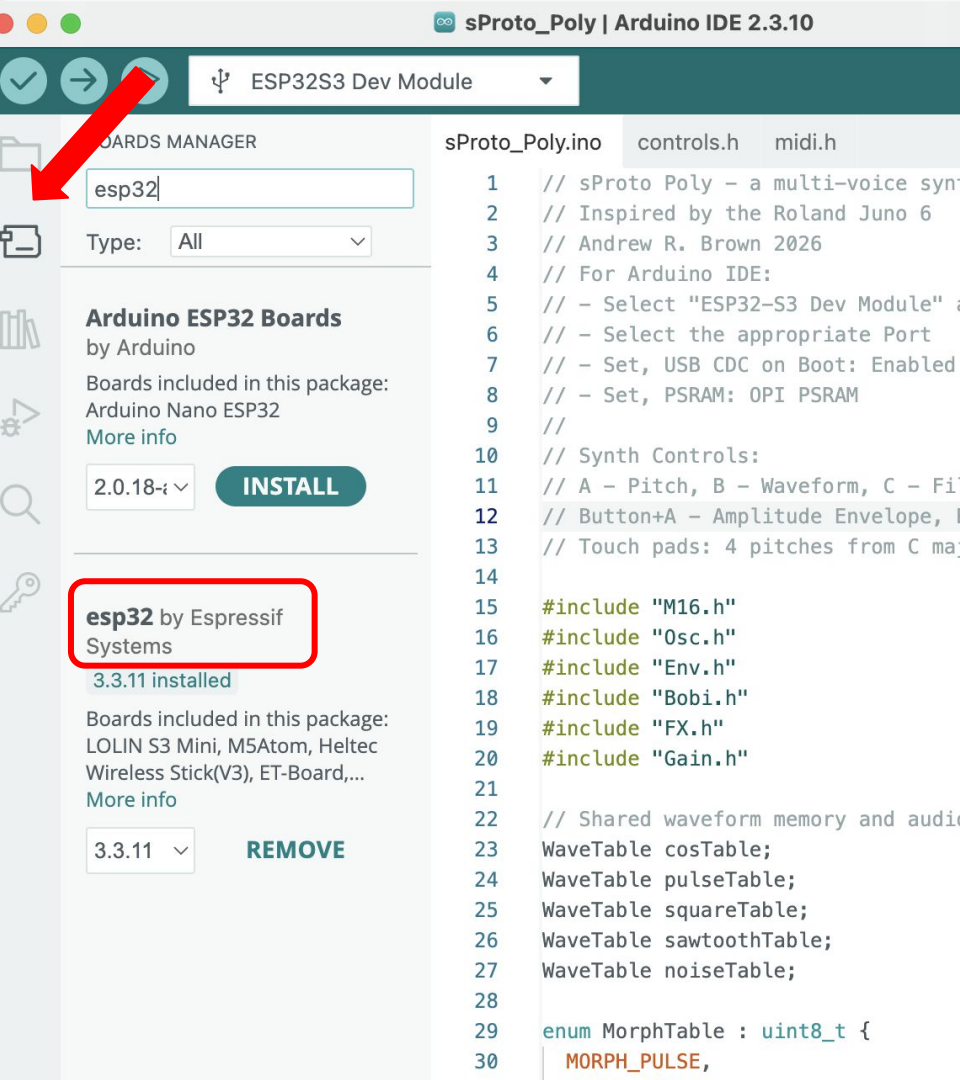
Add the ESP32 boards manager URL

Open Arduino IDE Settings

Google: installing esp32

Paste json link into

“Additional Boards Manager URLs”

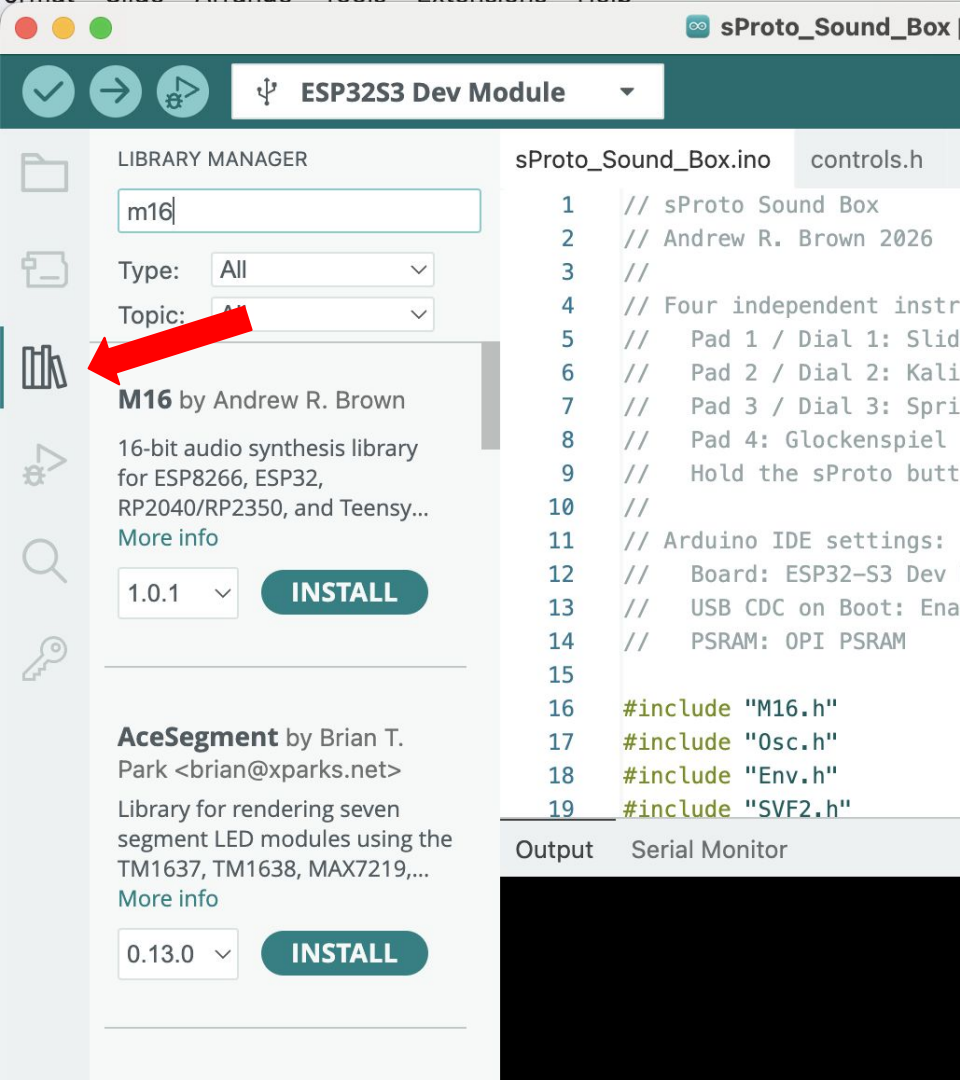


Add the ESP32 boards manager

Open Boards Manager (red arrow)

Locate ESP32 by Espressif

Click INSTALL



Install the libraries

Open the Library Manager (arrow)

M16

- <https://github.com/algomusic/M16>

MultiControl

- <https://github.com/algomusic/MultiControl>

M16 Library API Reference

A 16-bit audio synthesis library for ESP8266, ESP32, RP2040/RP2350 (Pico), and Teensy 4.x using I2S DACs and ADCs.

Quick Start

```
#include "M16.h"
#include "Osc.h"

Osc osc;

void setup() {
  Serial.begin(115200);
  setIsDualCore(false);
  osc.sinGen();
  osc.setPitch(69); // A4
  audioStart();
}
```

Pages 1

Home

M16 Library API Reference

Quick Start

Core (M16.h)

Audio System

Platform Behaviour

Dual-Core Partition API

Pico Family

Utilities

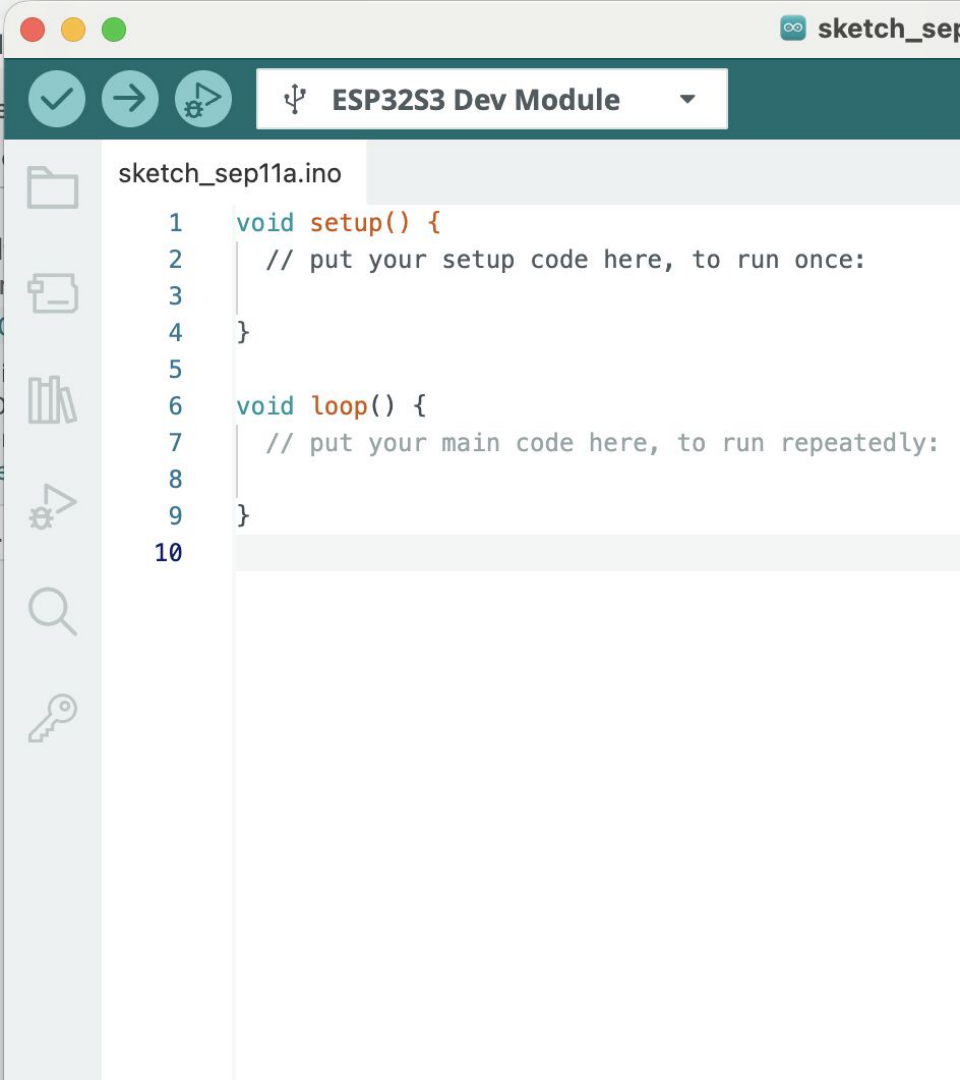
Osc.h — Wavetable Oscillator

Generating a Table

Sharing Tables

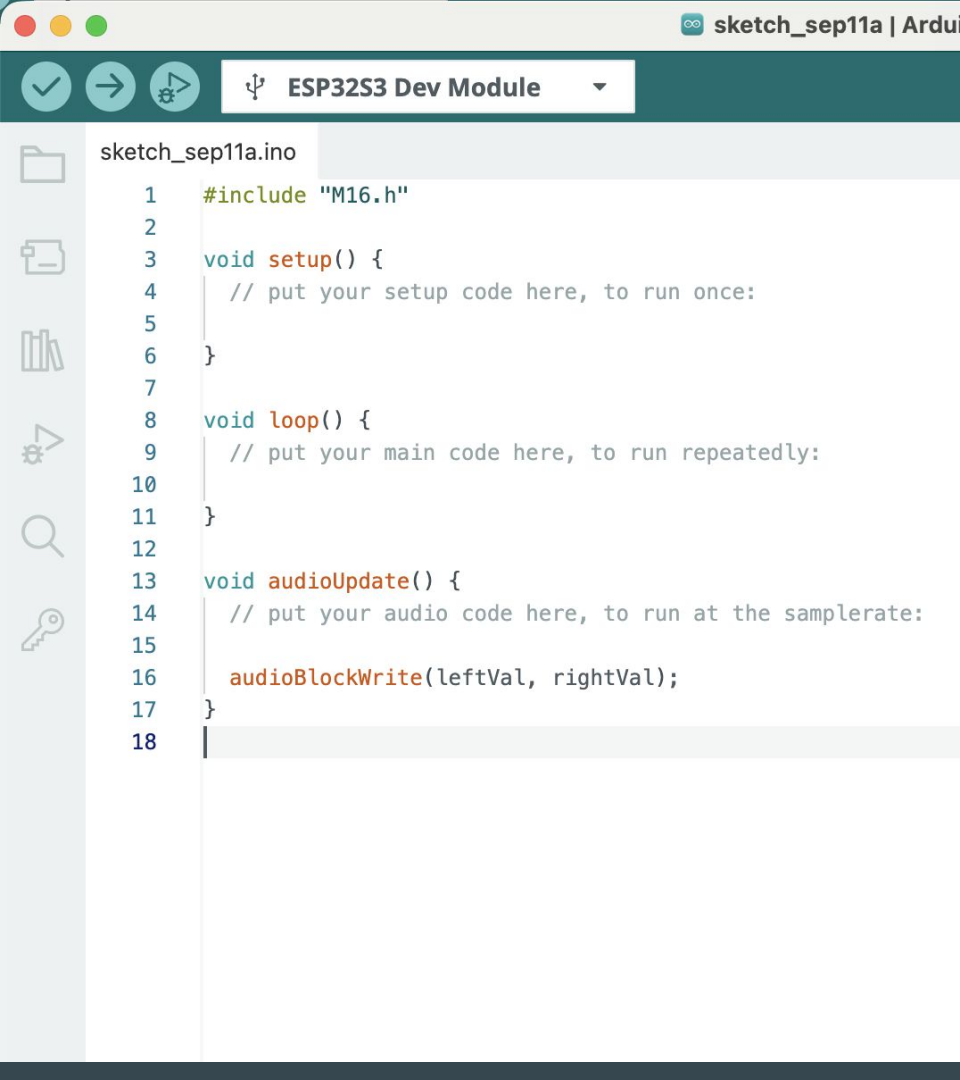
Key Methods

Modulation



New Sketch

```
void setup() {  
    // put your setup code here, to run once:  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
}
```



M16 Additions

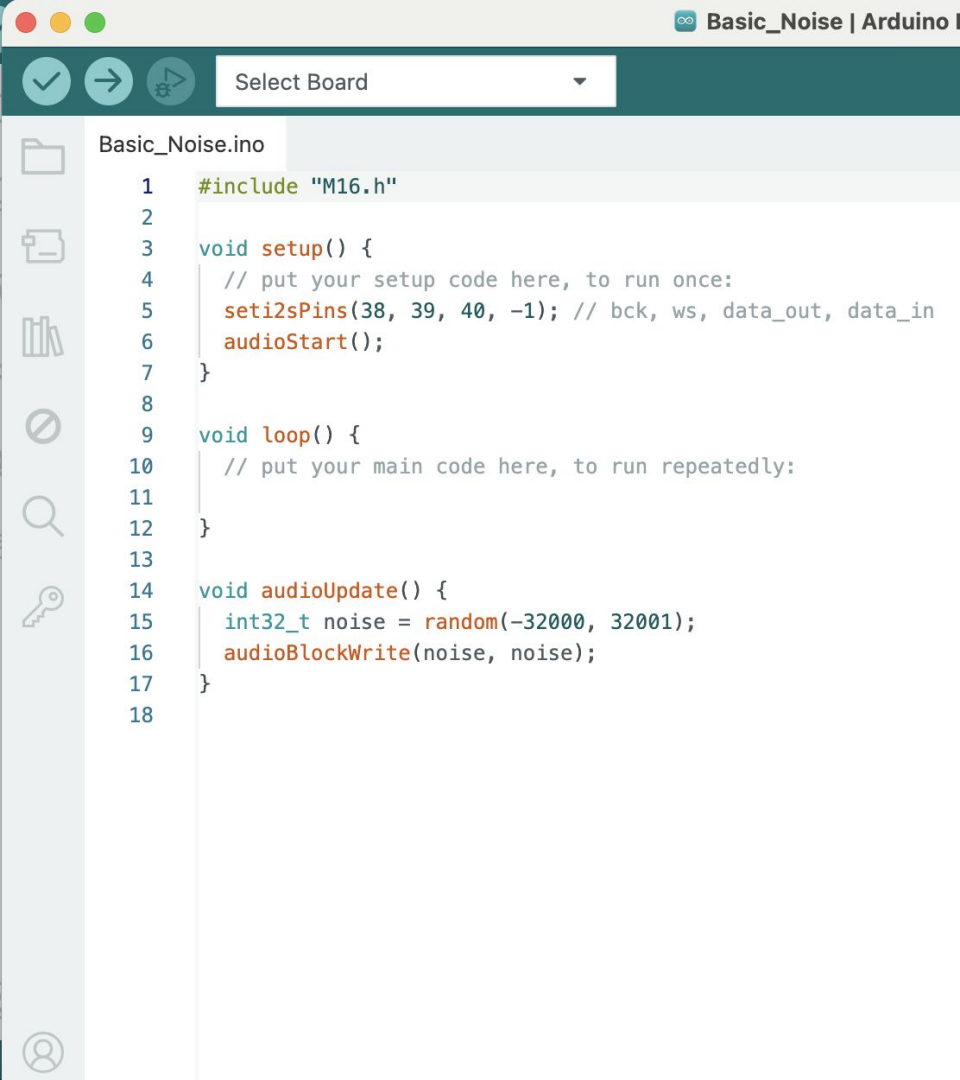
```
#include "M16.h"
```

```
void audioUpdate() {
```

```
    // put your audio code here, to run at the sample rate:
```

```
        audioBlockWrite(leftVal,  
rightVal);
```

```
}
```

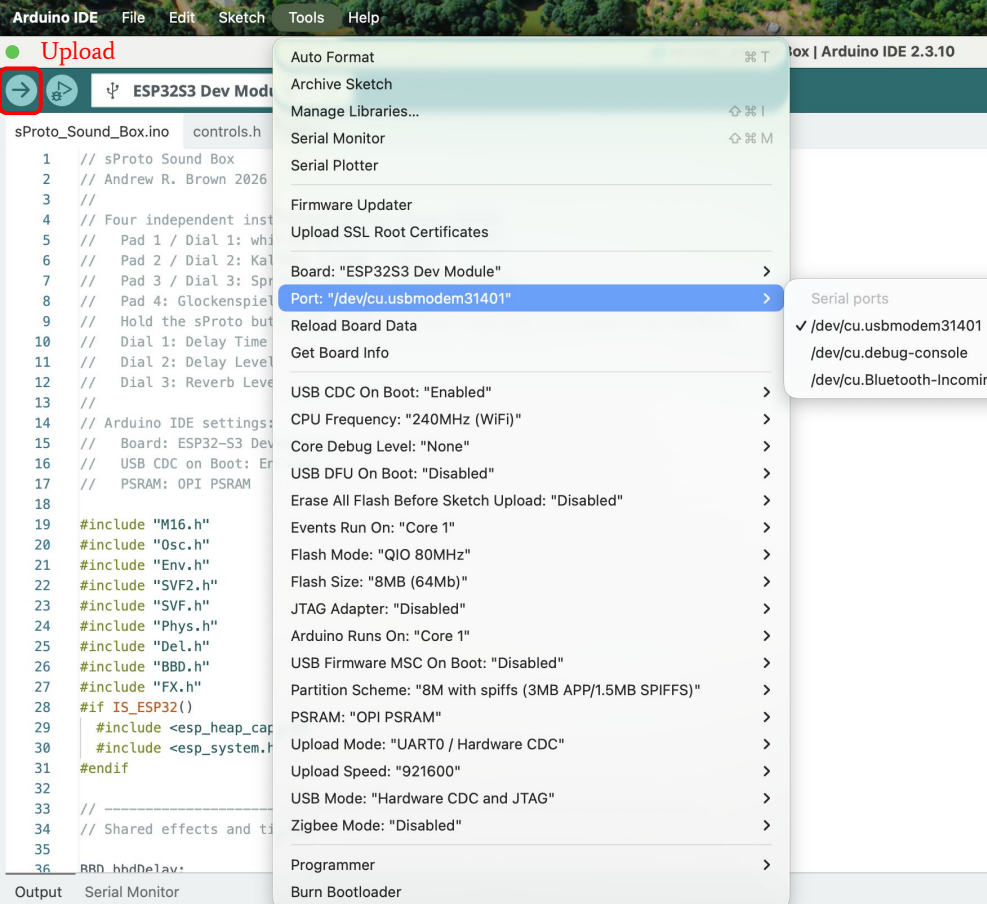


Basic Noise

```
seti2sPins(bck, ws, d_out, d_in);
```

```
audioStart();
```

```
int32_t noise = random(-32000, 32001);
```



Uploading

Select Board & Port from Tools menu

Upload (->)

algomusic / sProto

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

algomusic

sProto

Public

main

1 Branch

0 Tags

Go to file

algomusic

Initial commit

1d

Audio_Basics

Initial commit

Instruments

Initial Commit

.gitattributes

Initial commit

.gitignore

Initial Commit

LICENSE

Initial commit

README.md

Initial commit

README

CC0-1.0 license

sProto

Files that support using the sProto PCB

Files from GitHub

<https://github.com/algomusic/sproto>

sProto -> Code -> Download Zip

Extending Noise

- ▼  Audio_Basics
 - >  Basic_Noise
 - >  Basic_Filtered_Noise
 - >  Basic_Amp_Filtered_Noise
 - >  M16_SaH_Amp_Filter_Noise
 - >  Basic_SaH_Amp_Filtered_Noise
 - >  MControl_SaH_Amp_Filter_Noise

Filtering

Amplitude

S&H Noise

M16 classes

MultiControl

```

1  #include "MultiControl.h"
2
3  // sProto ESP32 Dev Module control pins
4  MultiControl dial1(5, 1); // GPIO, Potentiometer
5  MultiControl dial2(6, 1);
6  MultiControl dial3(7, 1);
7  MultiControl button(8, 2); // GPIO, Button
8
9  MultiControl pad1(1, 0); // GPIO, Touch
10 MultiControl pad2(2, 0);
11 MultiControl pad3(3, 0);
12 MultiControl pad4(4, 0);
13
14 void setup() {
15     Serial.begin(115200);
16     delay(500);
17     Serial.println("sProto MultiControl basics");
18 }
19
20 void loop() {
21     const int dial1Value = dial1.readPot();
22     const int dial2Value = dial2.readPot();
23     const int dial3Value = dial3.readPot();
24

```

MultiControl Basics

Initialise with

GPIO pin & type

MC Types:

Access values:

0 = Touch

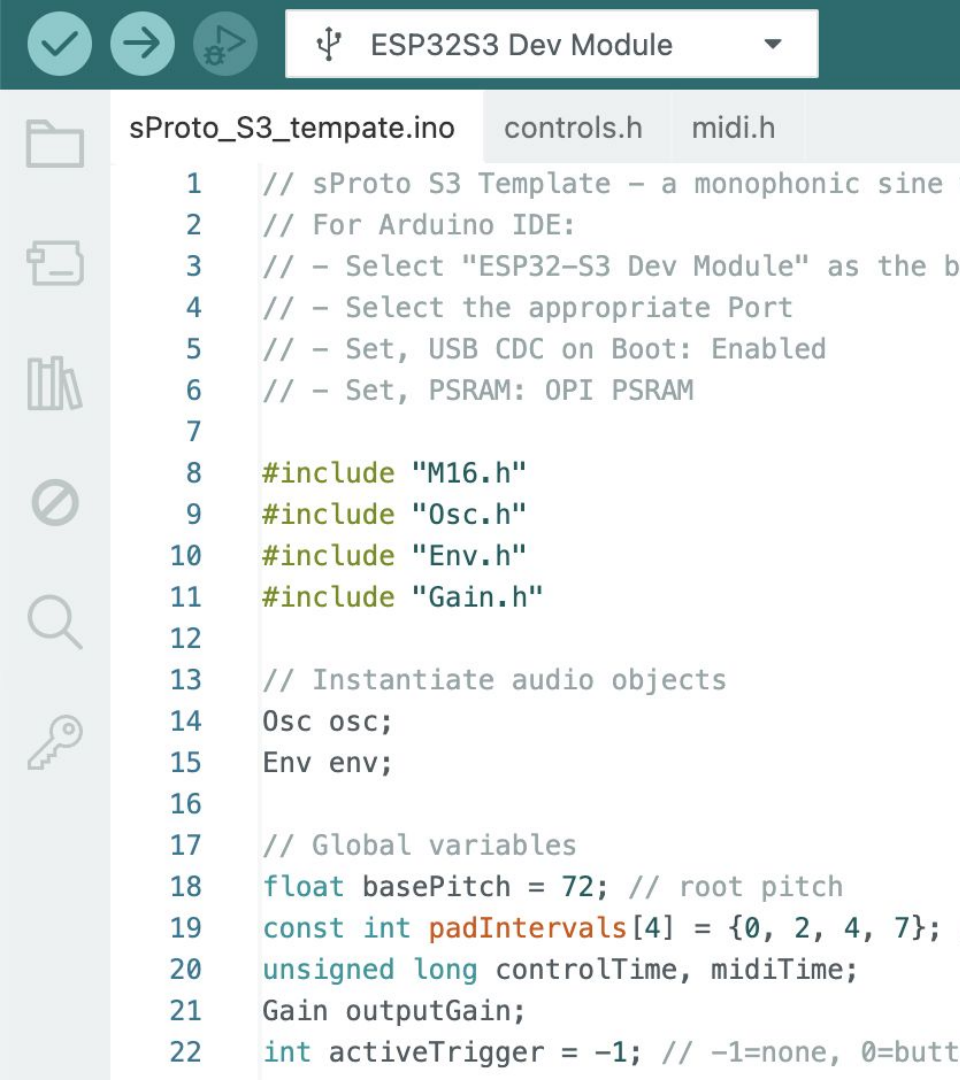
readPot() - int

1 = Potentiometer

isPressed() - boolean

2 = Button

isTouched() - boolean



sProto S3 Template

3 Files:

- template.ino
- controls.h
- midi.h

Music Box Instruments

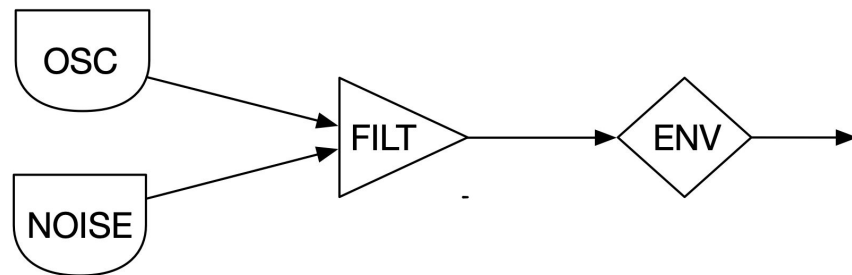
- Tin Whistle
- Glockenspiel
- Kalimba
- Spring



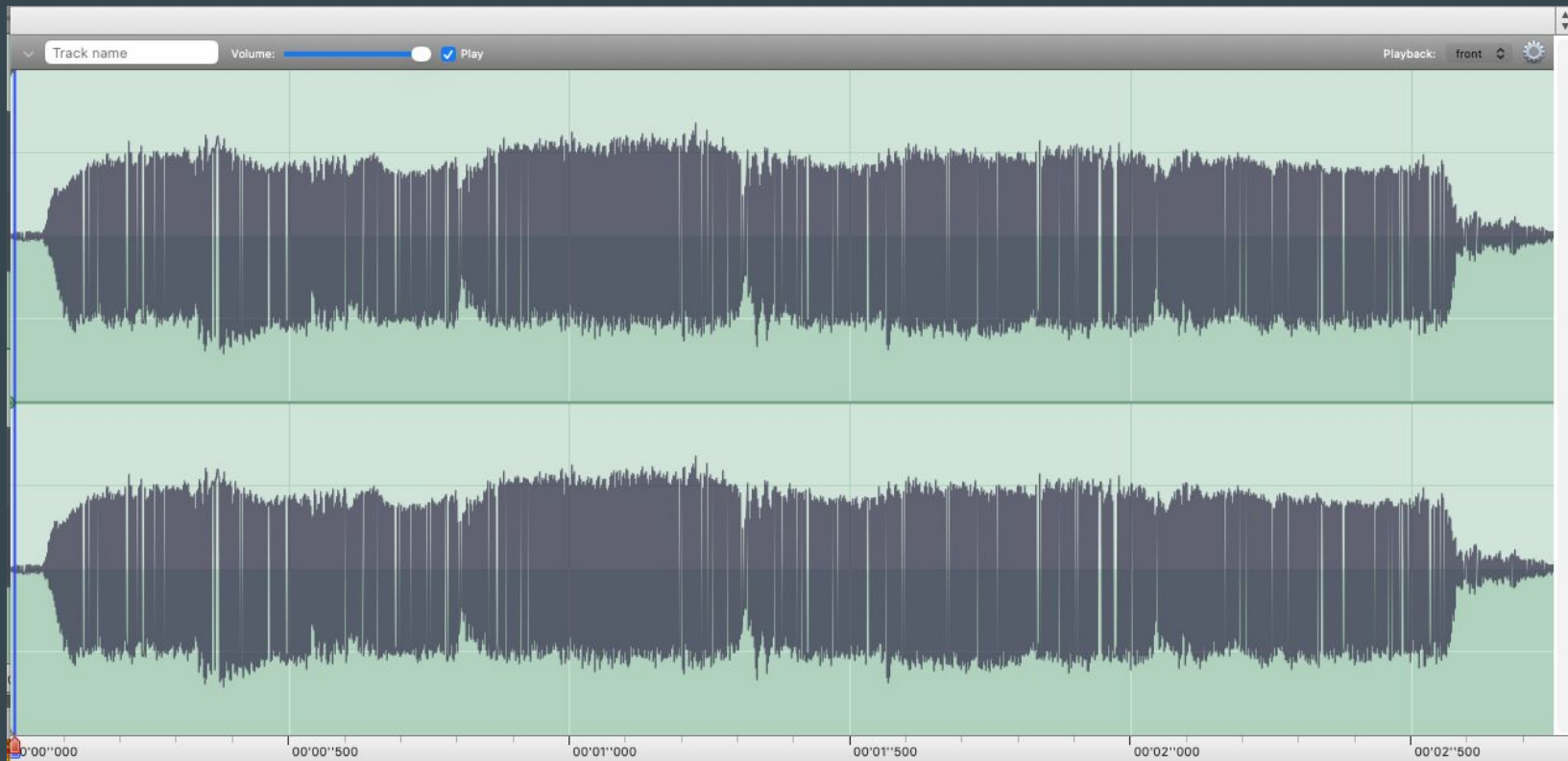


Tin Whistle

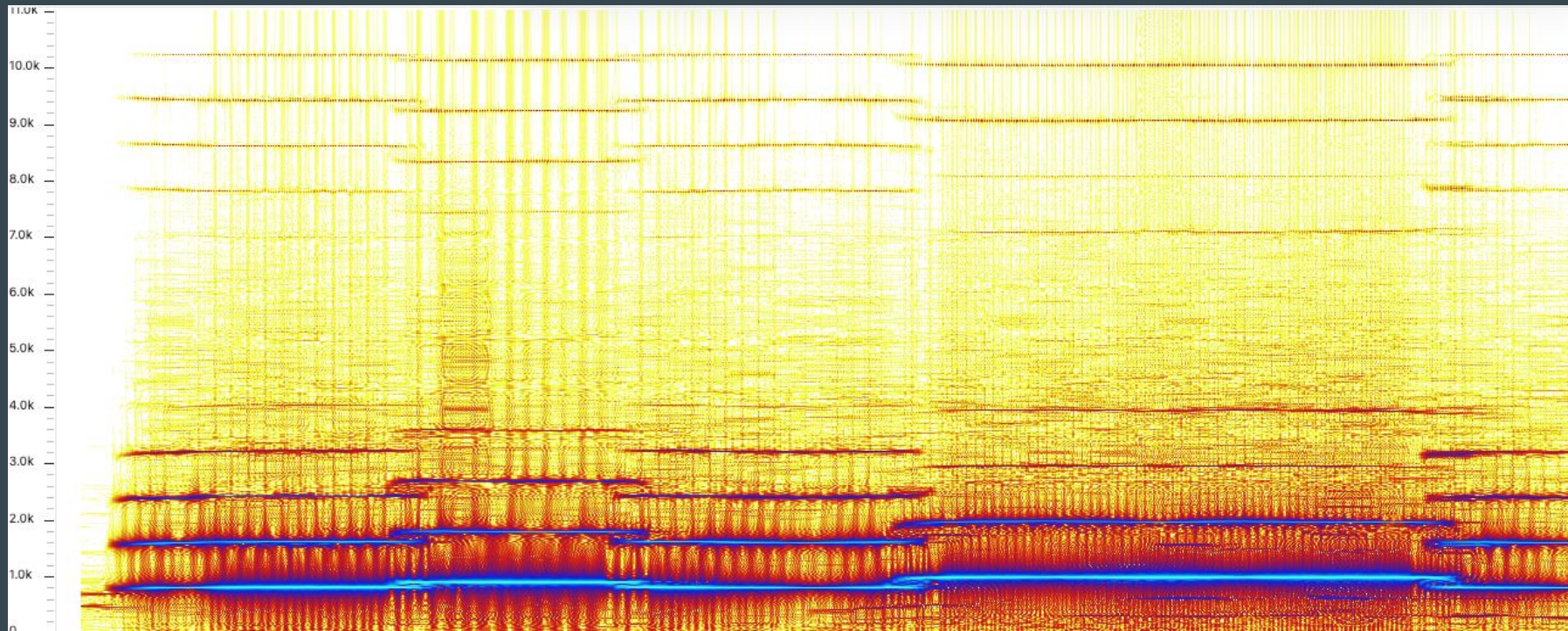
Subtractive Synthesis



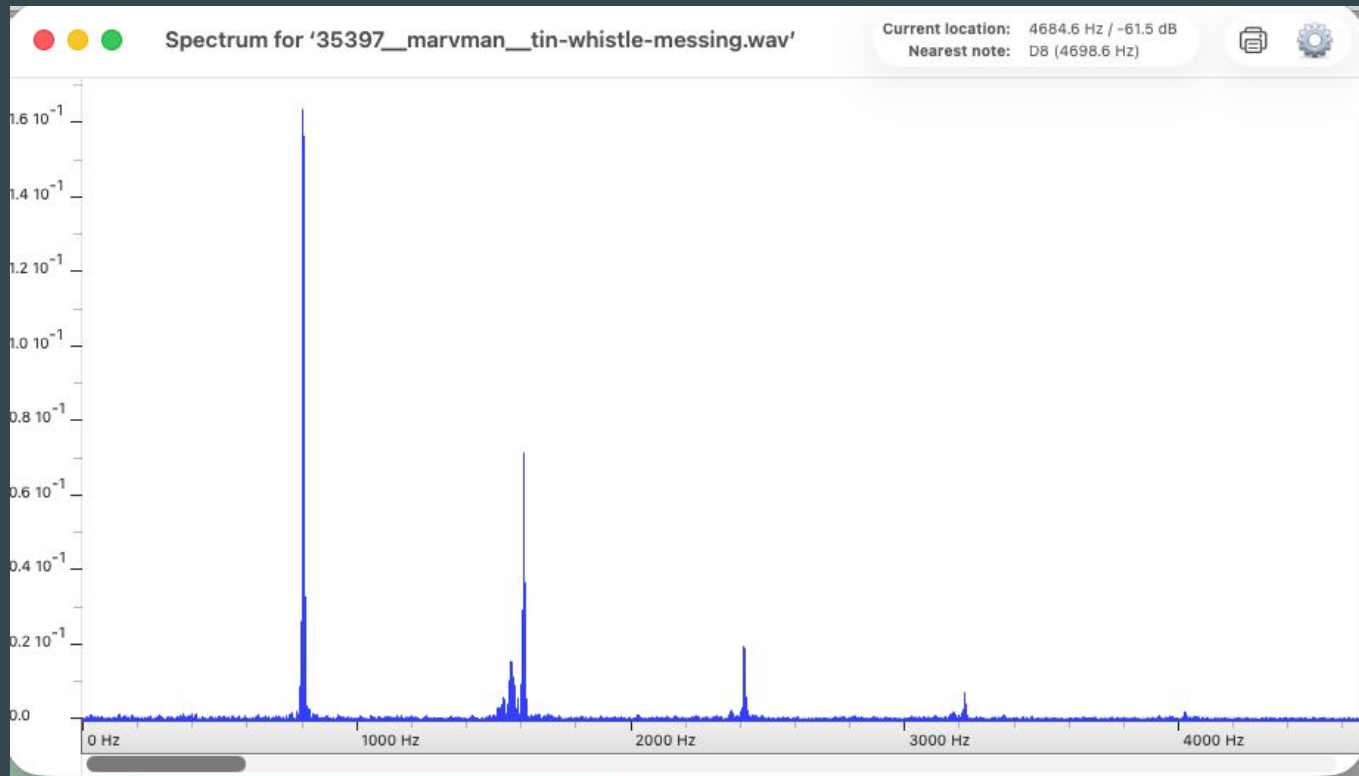
Tin Whistle Waveform



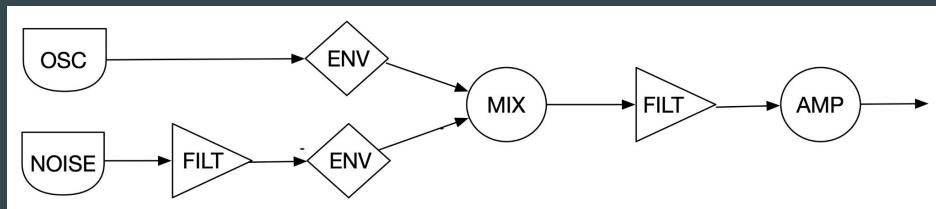
Tin Whistle Sonogram



Tin Whistle Spectrum



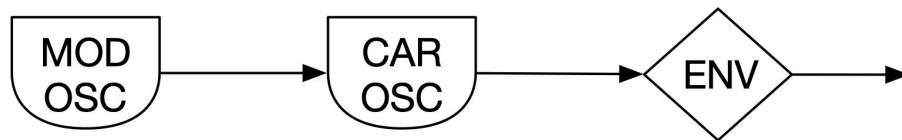
Tin Whistle synthesis function



```
/* The audioUpdate function is required in all M16 programs. */  
void audioUpdate() {  
    int32_t tone = osc.phMod(slideVibratoLfo.next(), slideVibratoPhaseIndex); // tone with vibrato  
    tone = (tone * toneAmpEnv.next()) >> 16; // apply amp env  
    int32_t wind = noise.next(); // generate noise  
    wind = noiseBPFILTER.nextBPF(wind); // bandpass filter the noise  
    wind = (wind * noiseAmpEnv.next()) >> 16; // amplitude envelope  
    int32_t out = clip16(tone + wind); // mix tone and noise  
    out = toneHPFILTER.nextHPF(toneLPFILTER.nextLPF(out)); // lowpass filter the mixed signal  
    out = outputGain.next(out); // volume level  
    audioBlockWrite(out, out); // send to DAC  
}
```

Glockenspiel

Frequency Modulation Synthesis



FM Spectrum

Sidebands at intervals of the modulating frequency symmetrical around the carrier frequency.

Roads, C. 1996. Computer Music Tutorial, The MIT Press.

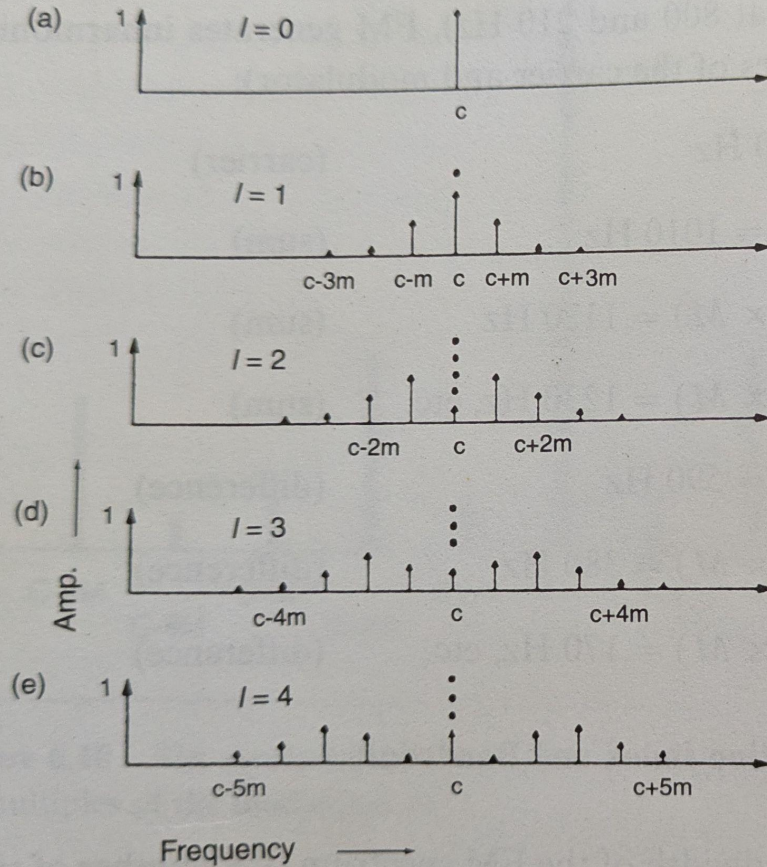
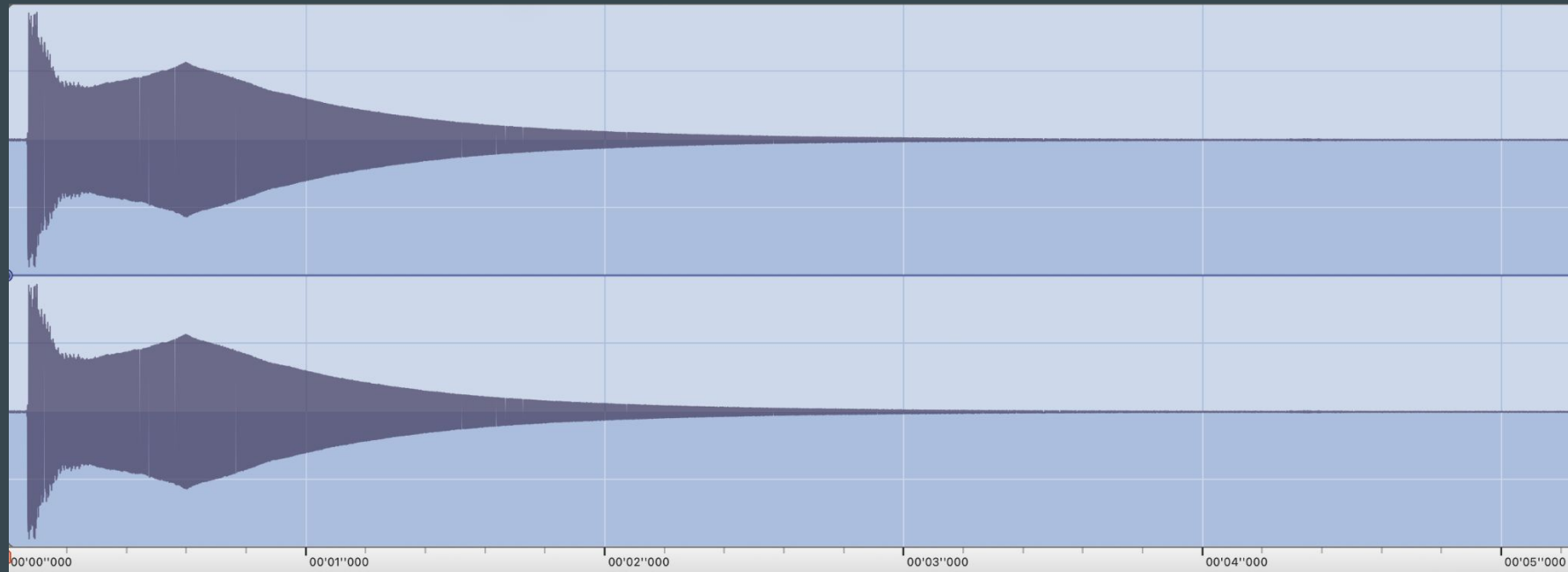
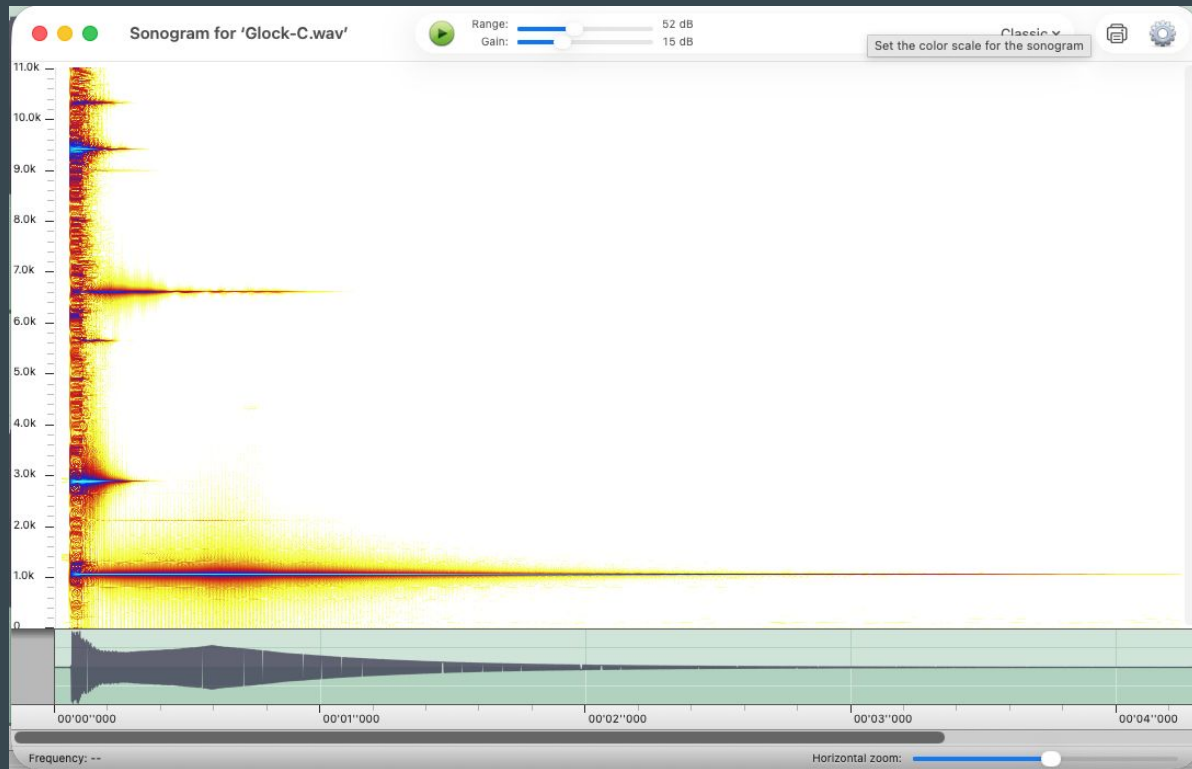


Figure 6.11 FM spectrum with increasing modulation index.

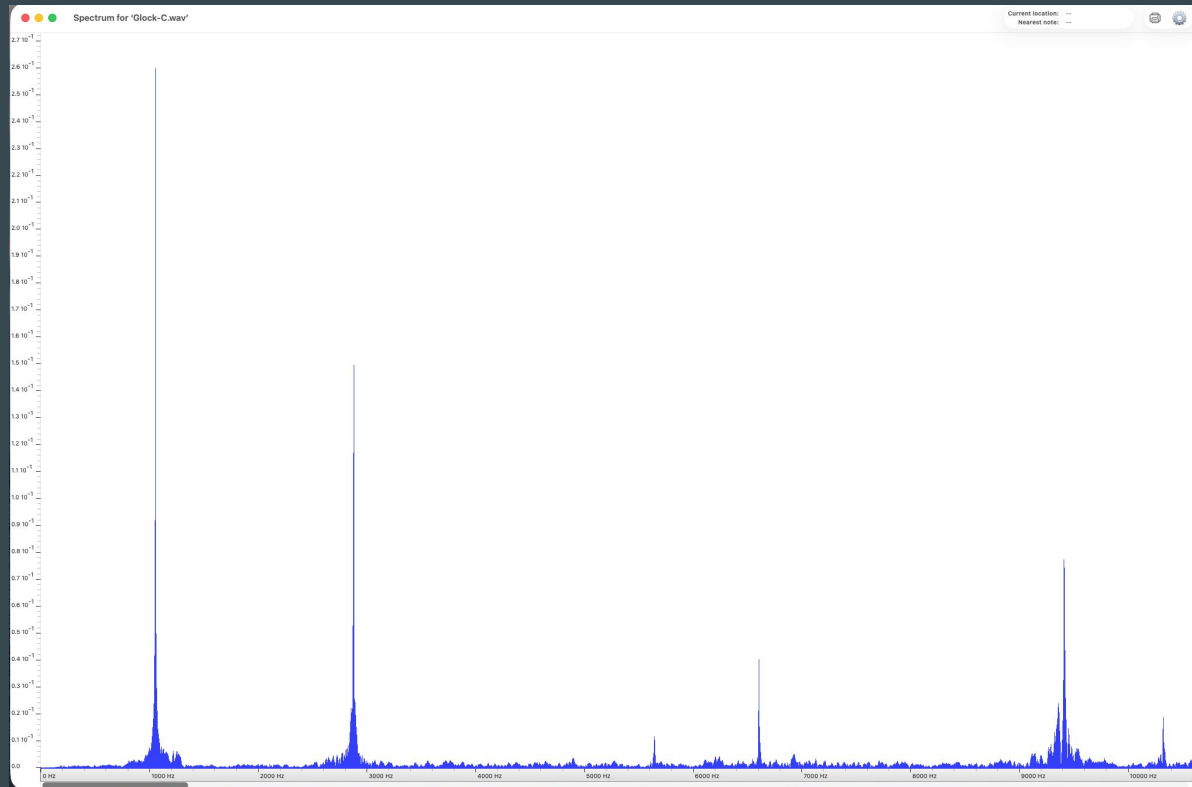
Glock Waveform



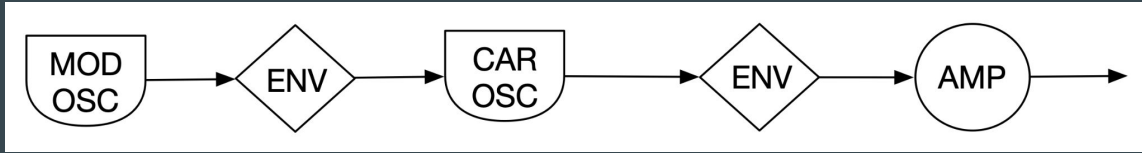
Glock Sonogram



Glock Spectrum



Glock synthesis function

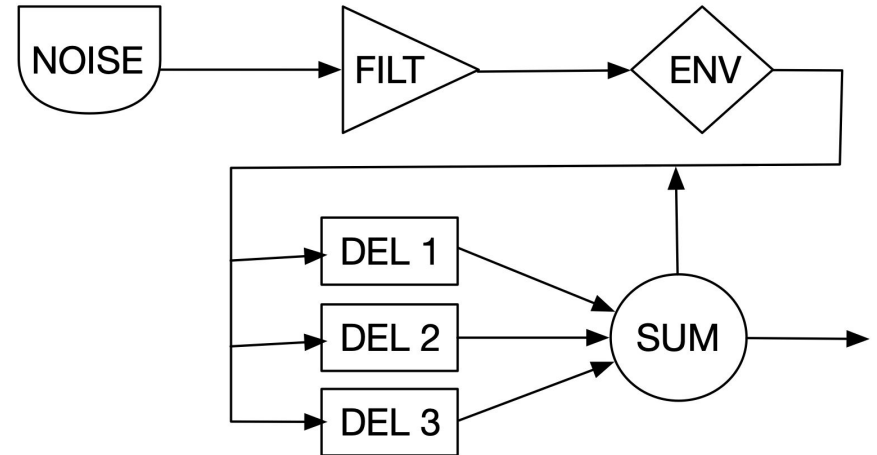


```
/* The audioUpdate function is required in all M16 programs. */  
void audioUpdate() {  
    int32_t modTone = modOsc.next(); // generate modulator  
    int32_t modAmpEnv = (modTone * modEnv.next()) >> 16; // shape modulator  
    int32_t tone = carrierOsc.phMod(modAmpEnv, modIndex); // generate modulated carrier  
    int32_t toneAmpEnv = (tone * carrierEnv.next()) >> 16; // shape the carrier  
    int32_t out = outputGain.next(toneAmpEnv); // volume  
    audioBlockWrite(out, out); // send to DAC, same signal to L & R  
}
```

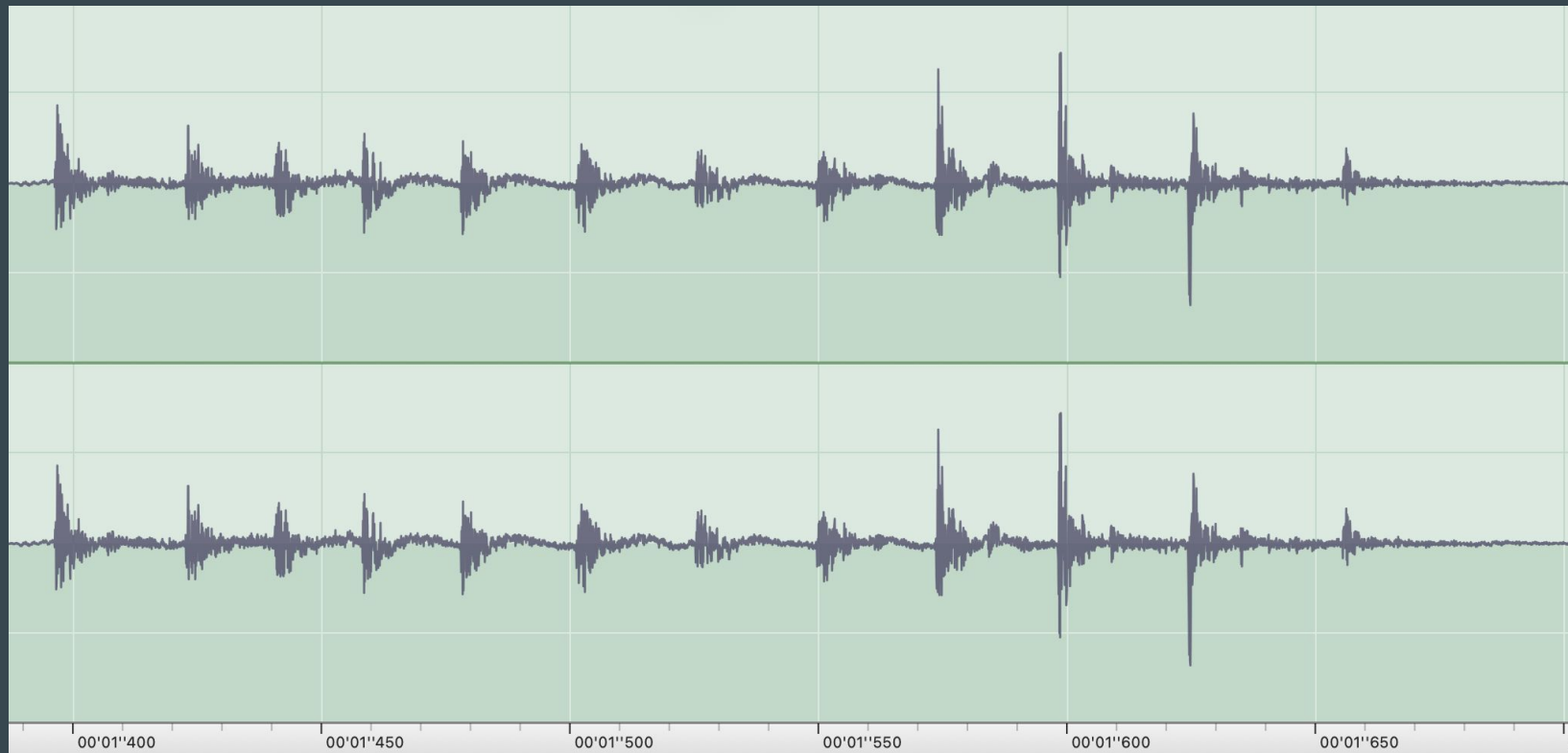


Spring

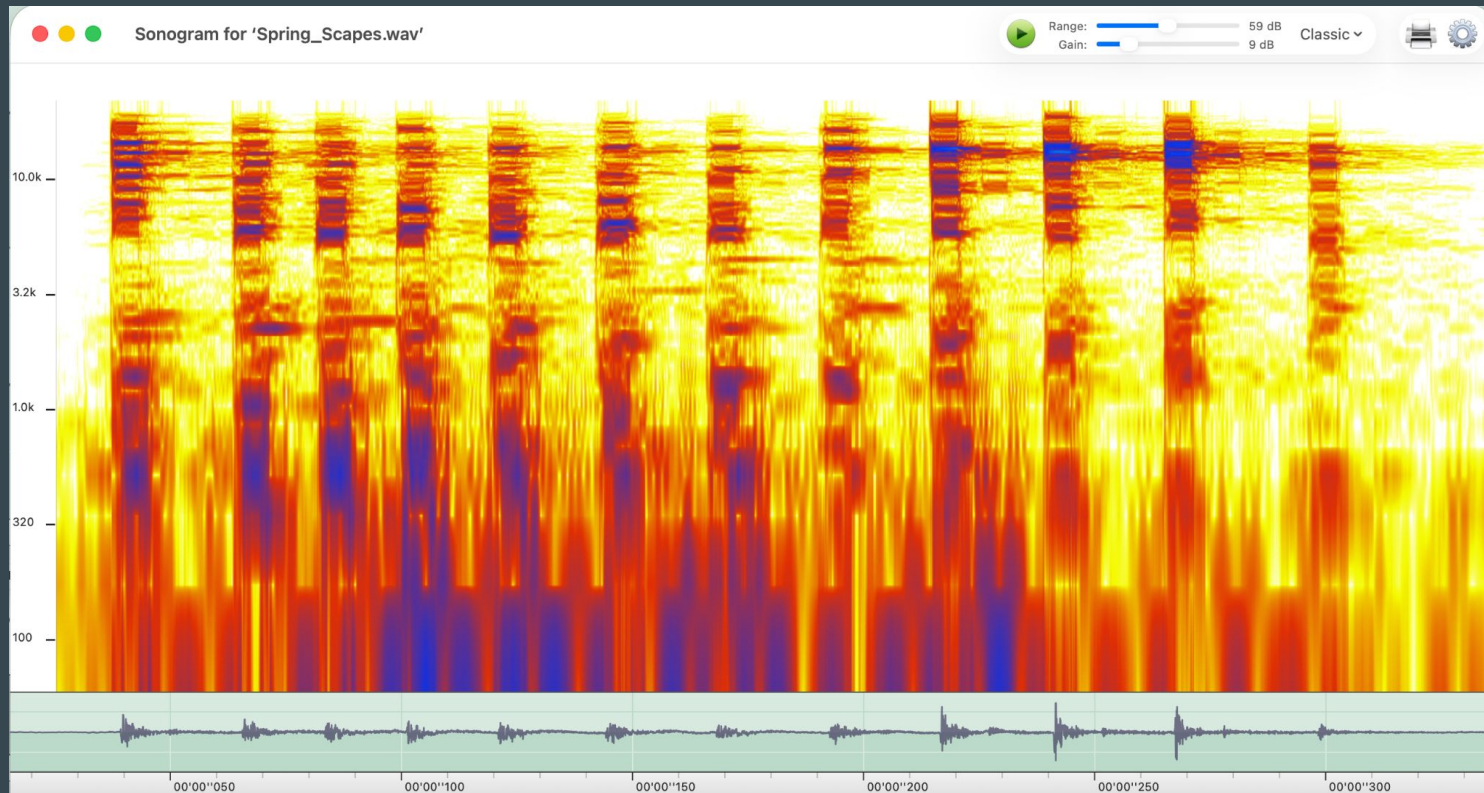
Excited Feedback Delays



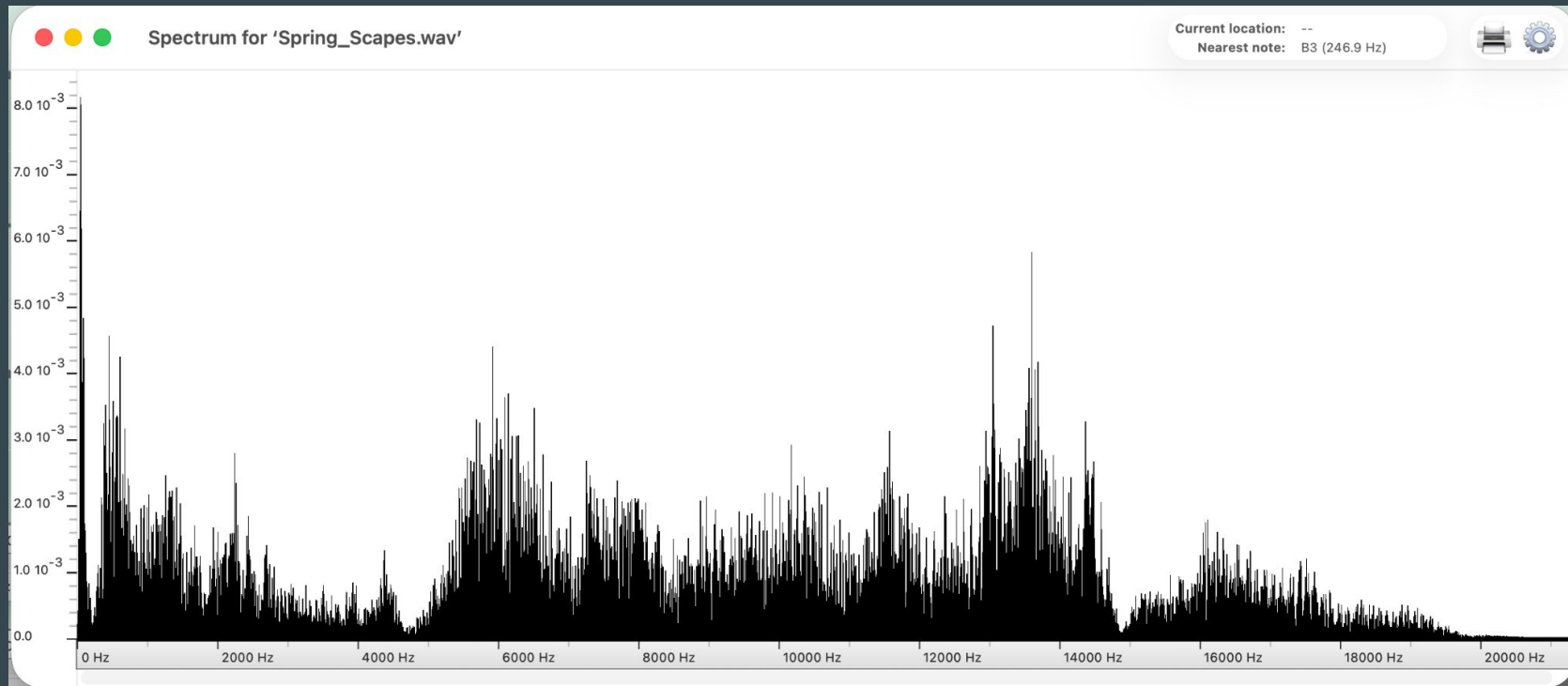
Spring Scrape Waveform



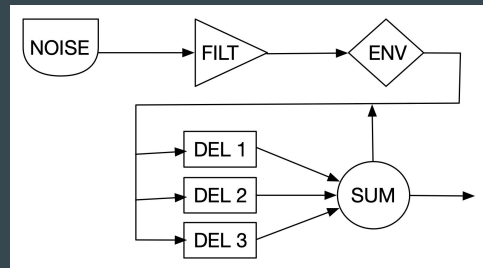
Spring Sonogram



Spring Spectrum



Spring synthesis function



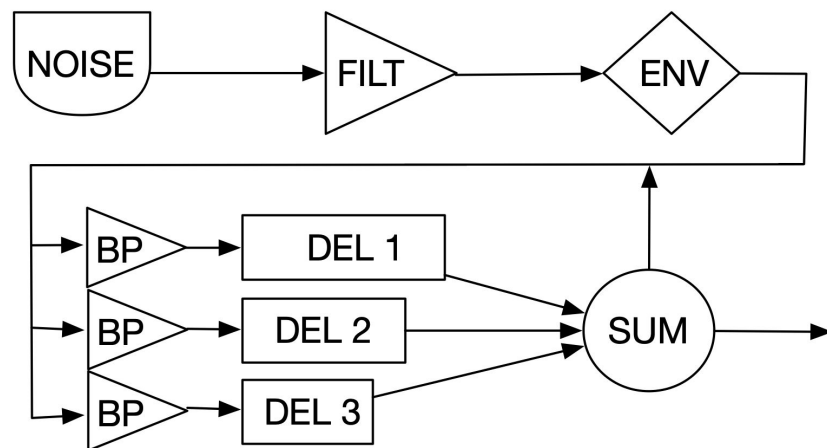
```
// --- audio ---
/* The audioUpdate function is required in all M16 programs. */
int32_t delaySum;

void audioUpdate() {
    int32_t tone = noise.next(); // generate noise exciter
    int32_t filteredTone = noiseFilter.nextBPF(tone); // bandpass filter the noise
    int32_t ampEnv = (filteredTone * env.next()) >> 16; // shape the noise into an impulse
    int32_t feed = clip16(ampEnv + (delaySum >> 6)); // add some cross mod between delays
    delaySum = clip16(delays[0].next(feed) + delays[1].next(feed) + delays[2].next(feed)); // sum three delay lines
    int32_t out = outputGain.next((ampEnv >> 1) + delaySum); // mix the exciter and delays
    audioBlockWrite(out, out); // send to DAC, same signal to L & R
}
```

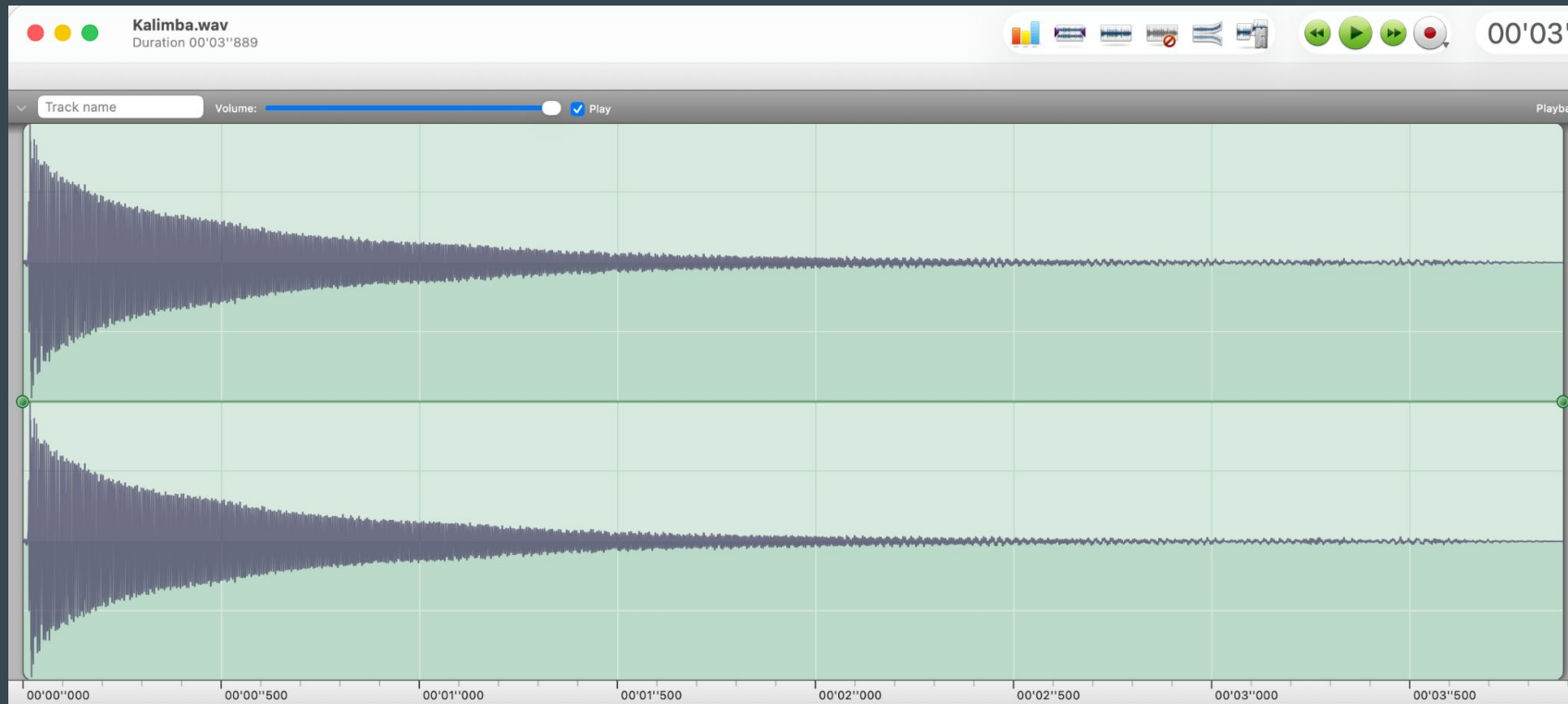


Kalimba

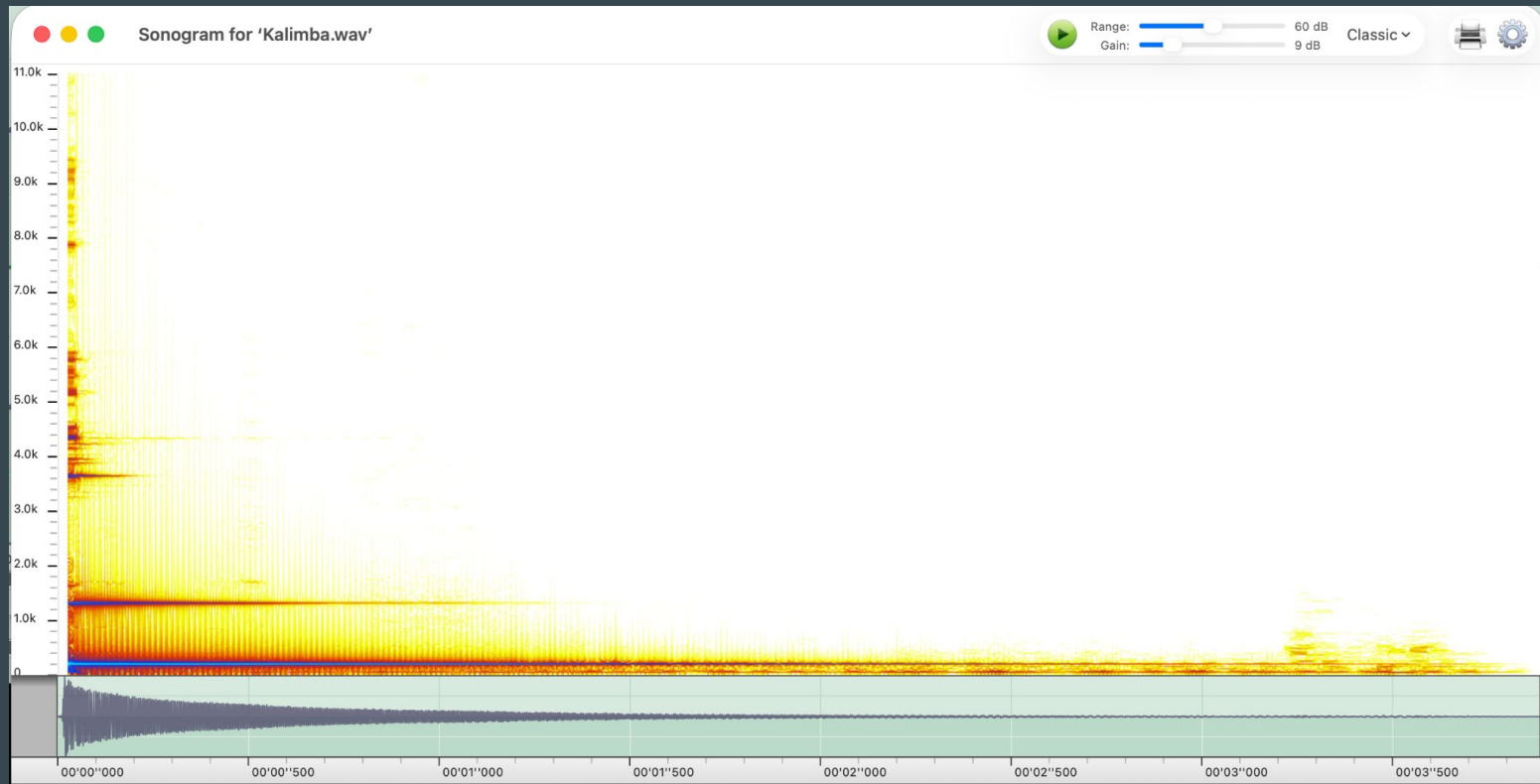
Banded Waveguide



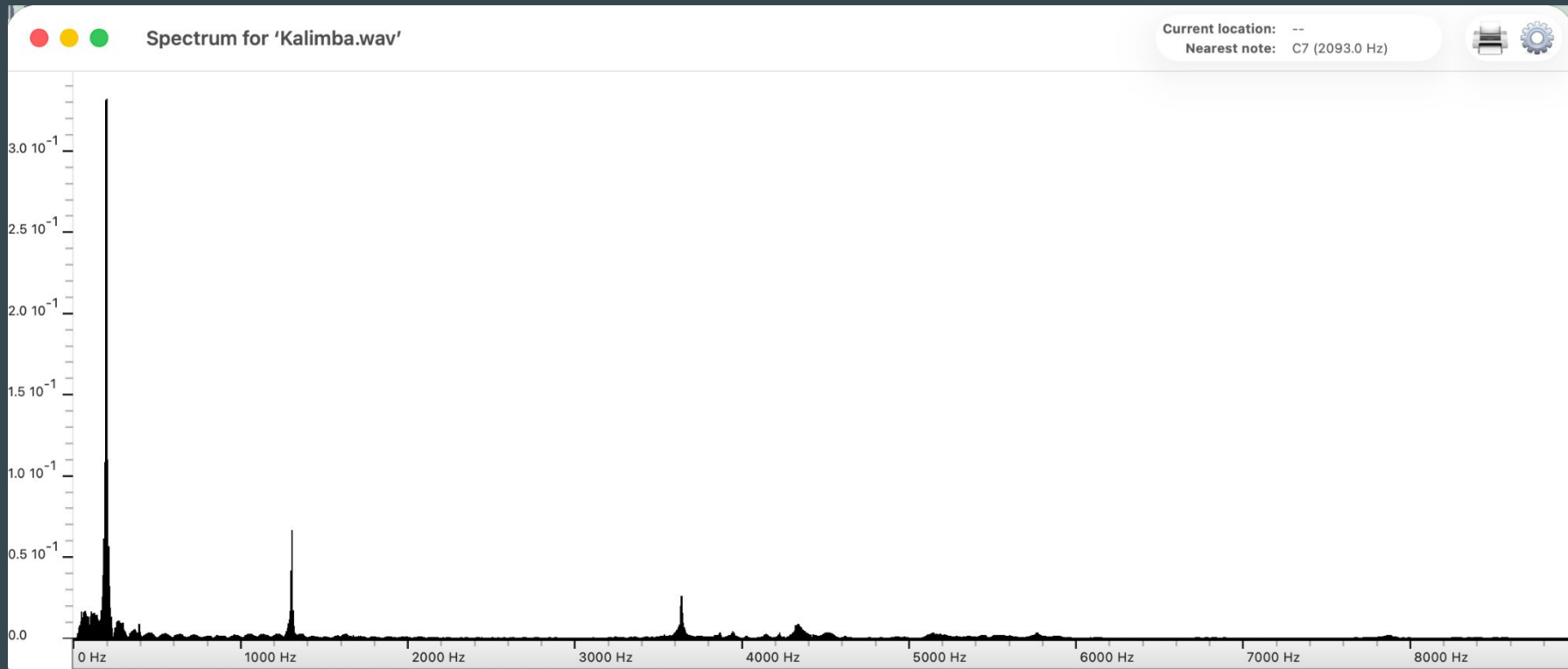
Kalimba Waveform



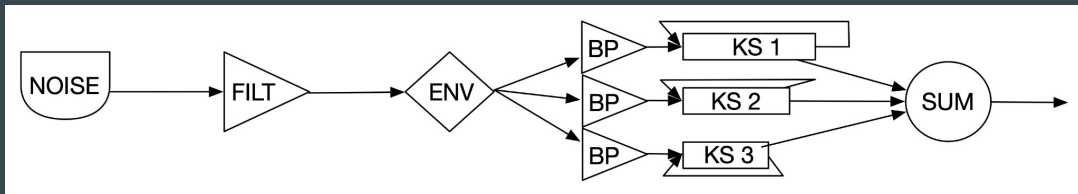
Kalimba Sonogram



Kalimba Spectrum

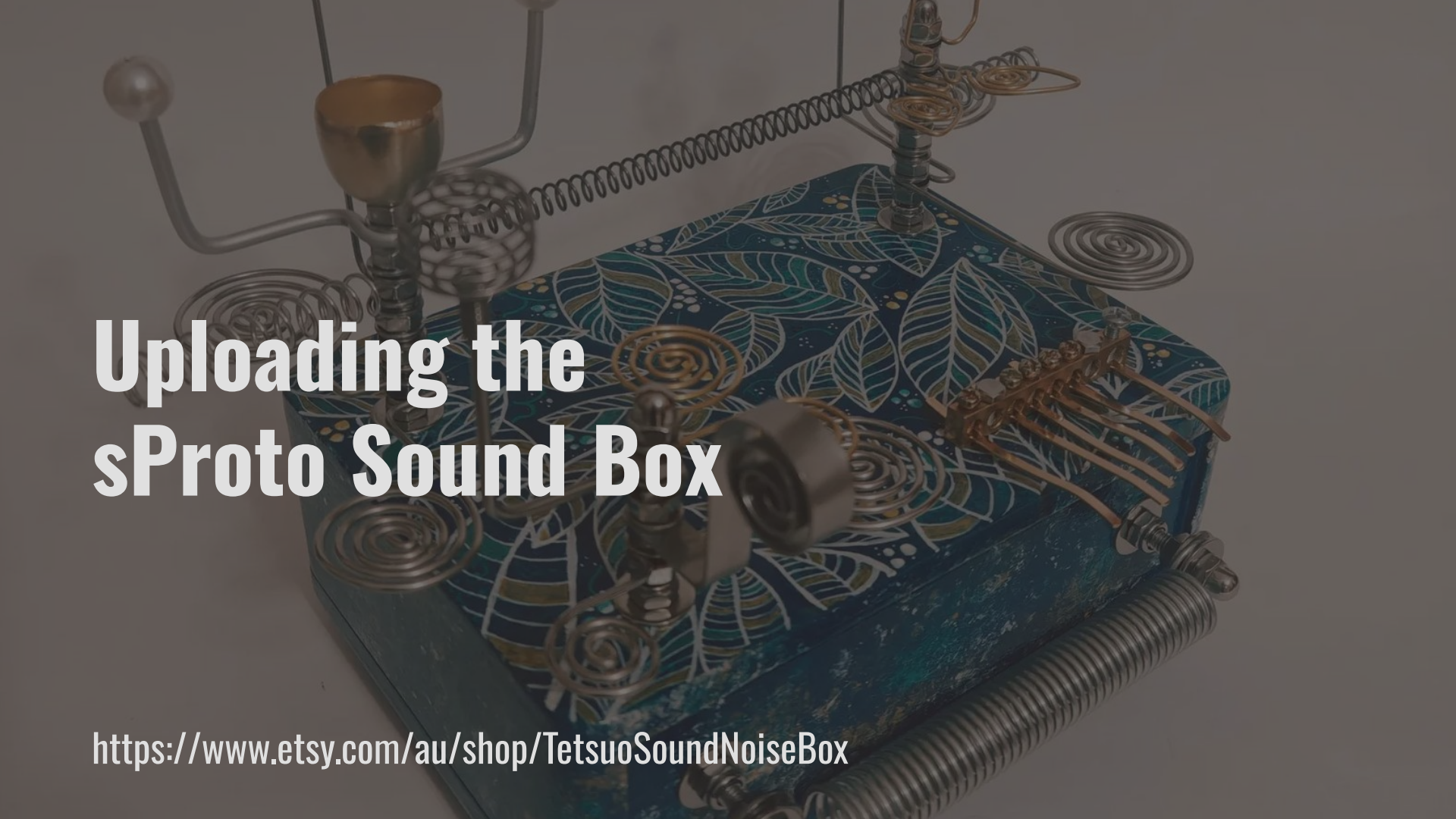


Kalimba synthesis function



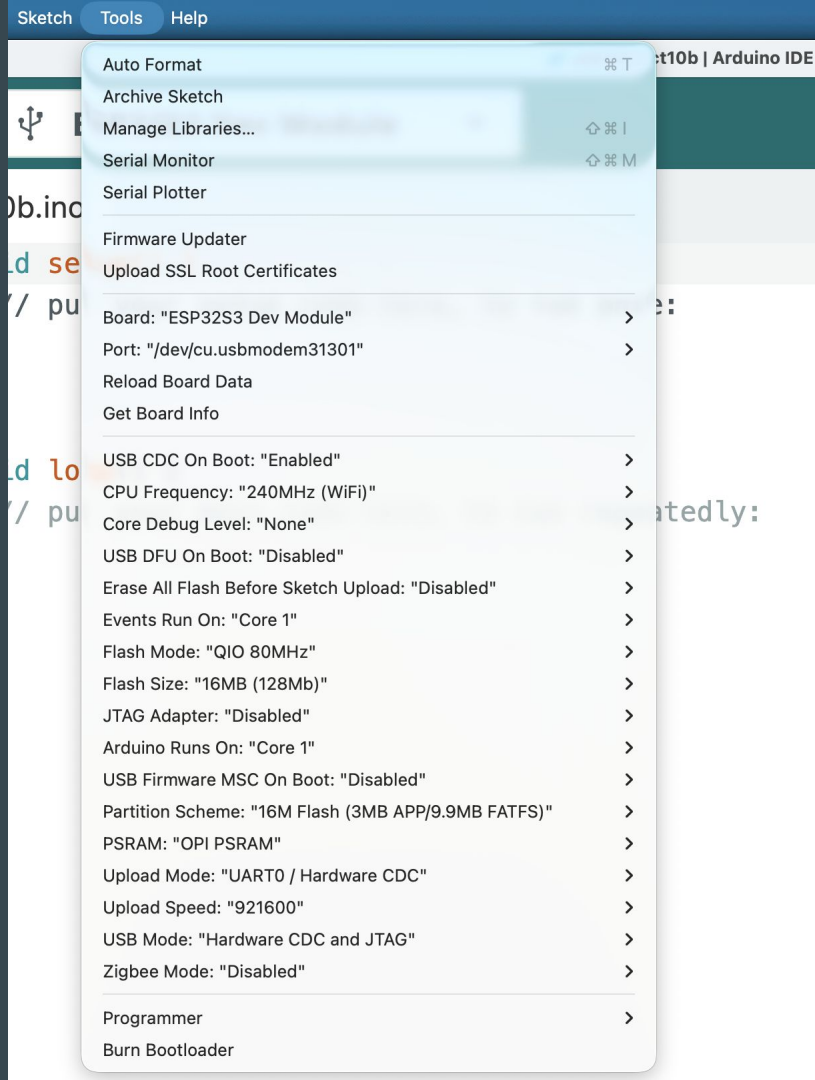
```
/* The audioUpdate function is required in all M16 programs. */
void audioUpdate() {
    int32_t mix = 0; // reused variable for the combined output of all polyphonic voices
    // polyphonic voices are distributed across the 2 cores of the ESP32
    for (int voice = audioPartitionOffset(); voice < voiceCount; voice += audioPartitionStride()) { // for each voice
        int32_t source = noise[voice].nextUnlocked(); // generate noise
        int32_t strike = ((int64_t)source * envelopes[voice].next()) >> 16; // shape noise into an exciter impulse
        int32_t dampenedStrike = exciterFilters[voice].nextLPFUnlocked(strike); // lowpass filter the noise
        // pass exciter through three bandpass filters, then to three tuned Karplus Strong feedback delay lines
        int32_t mode1 = modes[voice][0].pluck(modeFilters[voice][0].nextBPFUnlocked(clip16(dampenedStrike)), feedback1);
        int32_t mode2 = modes[voice][1].pluck(modeFilters[voice][1].nextBPFUnlocked(clip16(dampenedStrike)), feedback2);
        int32_t mode3 = modes[voice][2].pluck(modeFilters[voice][2].nextBPFUnlocked(clip16(dampenedStrike)), feedback3);
        // mix the components together, gain compensating to prevent clipping
        int32_t voiceMix = clip16(((dampenedStrike >> 1) + mode1 + (mode2 >> 1) + (mode3 >> 1)) >> 1);
        mix = clip16(mix + voiceMix);
    }

    audioBlockWrite(mix, mix); // send to the DAC, same mono signal to each L & R channel.
}
```



Uploading the sProto Sound Box

<https://www.etsy.com/au/shop/TetsuoSoundNoiseBox>



Arduino Settings

The sProto Sound Box sketch needs as much memory as it can get, so ensure these board settings are set in the Tools menu.

Flash Size: 16 MB

Partition Scheme: 16 M Flash

PSRAM: “OPI PSRAM”

sProto Sound Box

Four independent instruments on one sProto board:

- Pad 1 / Dial 1: Tin Whistle, Trigger / Pitch
- Pad 2 / Dial 2: Kalimba, Trigger / Pitch
- Pad 3 / Dial 3: Spring, Trigger / Multiple Triggers
- Pad 4: Glockenspiel, Trigger

Hold the sProto button to adjust effects:

- Dial 1: Delay Time
- Dial 2: Delay Level
- Dial 3: Reverb Level

Coding a Digital Sound Box

...

On the ESP32 with the sProto PCB & Arduino IDE